

AERIS Checkpoint — ML Audit, EDA, Feature Engineering, and Flyability-Aware Expansion

We are entering the **ML audit + trust-building phase** of AERIS.

The previous phase successfully validated the **full curated → promoted → grouped-split → train → compare ML workflow** on the large aerodynamic dataset, but we intentionally paused before claiming research-grade validity.

The new chat must continue from this exact point.

1) PROJECT CONTEXT — WHERE WE ARE IN THE ROADMAP

AERIS is a **modular BWB UAV conceptual design platform** with this validated pipeline:

geometry generation
→ deterministic sampled datasets
→ AVL/AeroSandbox aerodynamic sweeps
→ QC
→ curation
→ promotion gating
→ grouped ML split by geometry
→ surrogate model training
→ model comparison
→ future flyability-aware AI design exploration

The current active dataset is:

data/datasets/benchmark_aero_v1

This dataset has already been:

- curated successfully
- force-promoted (1 rejected geometry only: geom_00190)
- gated with --allow-forced
- loaded through training-data + grouped split
- used in ML training and comparison

Current promoted dataset stats:

- kept rows: **32292**
 - kept geometries: **299**
 - grouped split:
 - train: 22572
 - val: 4860
 - test: 4860
-

2) COMPLETED ML FOUNDATION WORK

The following ML workflow is already implemented and validated:

dataset side

- `training_data.py`
- grouped split by `geometry_id`
- promotion gate enforcement
- allow-forced override
- split row CSV artifact saving
- split hash consistency checks in `compare`

ML side

- `train.py`
- `compare.py`
- `predict.py`
- `model_registry.py`

Current supported models:

- `linear_regression`
- `ridge`
- `elastic_net`

- random_forest
 - extra_trees
 - gradient_boosting
 - hist_gradient_boosting
-

3) CURRENT VERIFIED RESULTS

Using grouped split by geometry_id:

Best current model

hist_gradient_boosting

Current test metrics:

- mean $R^2 \approx 0.985$
- mean RMSE ≈ 0.0176

This currently beats:

- linear regression
- ridge
- random forest
- extra trees

This is promising, but NOT yet considered fully trustworthy.

4) CRITICAL SUSPICIONS / AUDIT FINDINGS

The previous chat identified several important ML trust issues.

These are now the priority.

A) Missing EDA

We moved too fast into model training.

Before trusting ML, we must perform:

- target distributions
- per-geometry coverage
- alpha/control coverage
- duplicate checks
- constant-column checks
- correlation matrix
- outlier scan
- nonlinearity checks
- target vs alpha / control plots

This is the first priority.

B) Missing feature engineering

Current feature set is mostly raw geometry + flight-condition inputs.

We now need engineered physics-informed features like:

- `alpha_deg_sq`
- `abs_alpha_deg`
- `control_input_deg_sq`
- `alpha_x_control`
- maybe dynamic pressure proxy
- maybe aspect-ratio / sweep composites
- maybe trim-sensitive interaction terms

This must remain modular and reproducible.

C) Suspicious explainability bug

The tree feature-importance export currently duplicates the SAME shared importance vector under every target for native multi-output tree models.

This is misleading.

The new chat must audit and fix:

```
train.py -> _extract_feature_importances_payload()
```

Correct behavior:

- store shared importance once
- OR wrap per target only when estimator is truly per-target
- avoid fake target-specific explanations

This is an important trust-layer fix.

D) Missing preprocessing pipeline

Linear and regularized models currently train on raw features without scaling.

This weakens:

- coefficient interpretability
- ridge fairness
- elastic net stability

The new chat should refactor:

```
model_registry.py
```

to support sklearn Pipeline:

- StandardScaler + LinearRegression
- StandardScaler + Ridge
- StandardScaler + ElasticNet

Trees should remain raw.

E) One-split overconfidence

Current compare uses only one grouped split seed.

We now need:

- repeated grouped seeds
- OR

- grouped cross-validation

This is mandatory before trusting paper-level conclusions.

5) IMMEDIATE NEXT DEVELOPMENT PHASE

The next chat should execute this exact order:

PHASE 1 — EDA MODULE

Create a modular ML EDA utility layer, ideally:

`src/aeris/ml/eda.py`

Functions:

- summarize features
- summarize targets
- detect constant columns
- detect duplicates
- correlation analysis
- per-geometry row count stats
- target coverage vs alpha/control
- save plots + JSON summary

Optional CLI:

`aeris ml eda`

This must work directly on promoted datasets.

PHASE 2 — FEATURE ENGINEERING LAYER

Create:

`src/aeris/ml/feature_engineering.py`

This must:

- accept DataFrame

- add engineered columns deterministically
- return transformed DataFrame + manifest of engineered features
- be reusable in train / compare / predict

No leakage.

Only physics-derived transformations from available inputs.

PHASE 3 — TRAINING PIPELINE REFACTOR

Refactor `train.py` so the workflow becomes:

load promoted dataset

→ EDA checks

→ optional feature engineering

→ split

→ preprocessing pipeline

→ train

→ evaluate

→ save explainability correctly

Keep backward compatibility.

PHASE 4 — MODEL REGISTRY EXPANSION

After trust fixes:

add external boosted models cleanly through registry seam:

priority:

1. XGBoost
2. LightGBM
3. optional CatBoost

Do NOT break registry architecture.

All model branching must remain centralized.

6) PAPER ROADMAP ALIGNMENT — FUTURE TARGETS

This ML phase is directly linked to the PhD roadmap.

Paper 2 requires:

AI surrogates for:

- CL
- CD
- Cm
- trim feasibility
- static stability
- control feasibility
- dynamic acceptability
- damping / eigen summaries

So after aerodynamic trust is established, the next target expansion should be:

near-term extra regression targets

- cn
- cy
- cl_roll
- x_np

later flyability-aware targets

- trim alpha
- trim control deflection
- static margin
- short-period metrics
- phugoid metrics
- spiral metric
- feasibility classification labels
- controllability labels

This must align with:

geometry → flyability-aware feasibility AI

not vanity aero regression.

7) EXPECTED OUTPUT OF THE NEXT CHAT

The next chat should NOT re-explain previous work.

It should immediately start by:

1. auditing ML flaws
2. creating the EDA module
3. creating feature engineering
4. fixing explainability bug
5. adding preprocessing pipelines
6. preparing repeated grouped validation
7. preparing future dynamics-target extension

Maintain AERIS architecture:

registry-driven, modular, CLI-thin, reproducible, promotion-gated, grouped-split safe.

This is checkpoint transition from:

ML foundation

to

ML trust + paper-grade surrogate science